

实验 3：基于 Shamir 秘密共享的安全多方平均值计算

数据安全第三次实验报告

姓名：郭佳成 学号：2311990 专业：密码科学与技术

代码：<https://github.com/Fighting05/DataSecurity>

2026 年 5 月 20 日

1 实验要求

参考教材实验 4.3.3（投票统计实例），实现三个人对他们拥有的数据的平均值的计算。

场景描述：假设有三个同学 Student1、Student2 和 Student3 各持有一个私有整数数据，希望在不向对方（以及计算方）泄露各自数据的前提下，共同计算这三个数据的平均值。

2 实验环境

- 操作系统：Windows 11
- 开发语言：Python 3.9
- 依赖库：无需额外安装，使用 Python 标准库

3 实验原理

3.1 安全多方计算概述

安全多方计算（Secure Multi-Party Computation, MPC）允许多个参与方联合计算一个函数，同时各方的私有输入不会被泄露。数据方持有私有数据，通过交互协议传给计算方，计算完成后结果方得到输出。整个过程中，任何一方都无法得知其他方的原始数据。

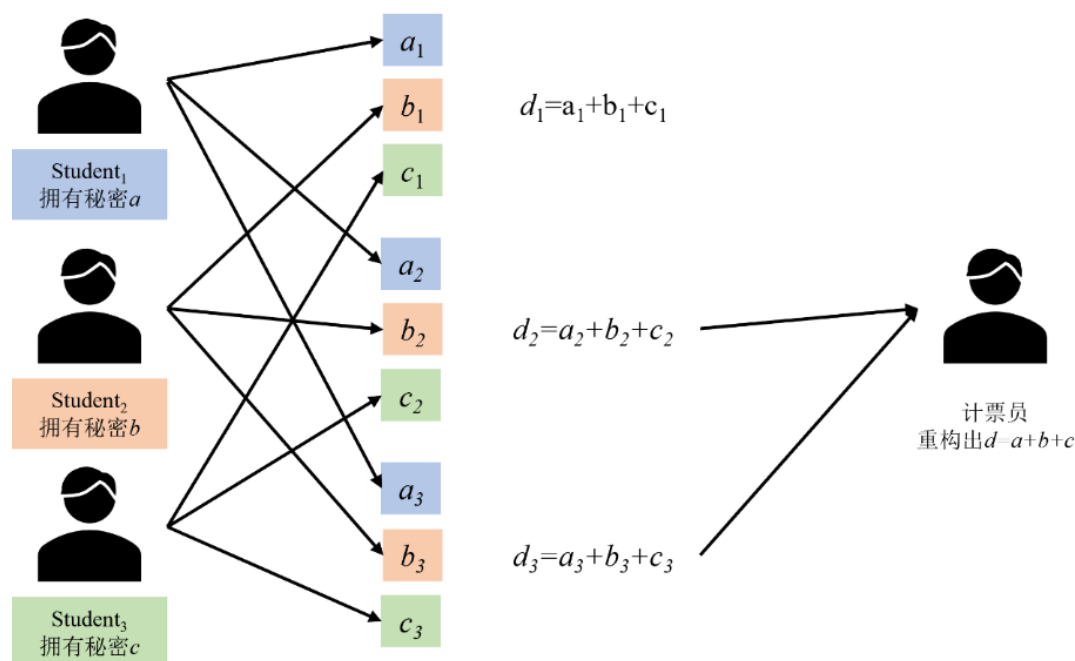


图 2: 协议流程: 三个学生各将私有数据用 (2,3) 门限 Shamir 分享, 计票员重构总和后求平均值

协议分四个步骤:

步骤 1 一秘密分发: 每个学生 i ($i = 1, 2, 3$) 把自己的私有数据 s_i 用 (2,3) Shamir 秘密共享发给全部三方。构造随机一次多项式 $f_i(x) = s_i + r_i \cdot x \pmod{p}$, 算三份份额:

$$(j, f_i(j)), \quad j = 1, 2, 3$$

将 $f_i(j)$ 发给 Student _{j} , 存为 $student_i_j.txt$ 。

步骤 2 一本地聚合: 每个学生 j 收集所有人给自己的份额, 相加:

$$d_j = f_1(j) + f_2(j) + f_3(j) \pmod{p}$$

设总和多项式 $F(x) = f_1(x) + f_2(x) + f_3(x)$, 则:

$$F(x) = (s_1 + r_1x) + (s_2 + r_2x) + (s_3 + r_3x) = (s_1 + s_2 + s_3) + (r_1 + r_2 + r_3)x$$

代入 $x = j$ 得 $F(j) = f_1(j) + f_2(j) + f_3(j) = d_j$, 即每个学生的聚合值 d_j 恰好是 F 在点 j 处的函数值。

步骤 3 一重构总和: 计票员取任意两个学生 (本实验取学生 2 和学生 3) 的聚合值 d_2, d_3 , 用拉格朗日插值恢复 $F(0) = s_1 + s_2 + s_3$ 。

步骤 4 一求平均值:

$$\text{avg} = (s_1 + s_2 + s_3) \times 3^{-1} \pmod{p}$$

$3^{-1} \pmod{p}$ 是 3 的乘法逆元, 由费马小定理 $3^{-1} \equiv 3^{p-2} \pmod{p}$ 计算。

关于”除以 3”：有限域 $\mathbb{Z}_p$ 里”除以 3”等价于”乘以 3 的乘法逆元 $3^{-1} \bmod p$ ”，不是整数除法。 $p = 1000000007$ 时 $3^{-1} \equiv 333333336 \pmod{p}$ 。如果数据之和能被 3 整除，模意义和整数意义结果一致；否则是模下的”等价值”。

与投票统计的关系：协议结构和教材 4.3.3 完全一致——秘密分发、本地聚合、拉格朗日重构，代码框架直接复用。唯一区别是最后多了求平均值的步骤，即把总和乘以 3 的乘法逆元。

3.4 本实现与教材的对应关系

教材（投票统计 4.3.3）	本实现（平均值计算）	说明
ss_student.py 输入投票值	ss_student.py 输入私有数据	接口相同，语义扩展
count_student.py 本地聚合	compute_local.py 本地聚合	逻辑完全一致
vote_counter.py 重构票数	average_counter.py 重构并求均值	增加除以 3 的步骤
无	run_test.py 自动化测试	新增，方便验证

4 代码实现

4.1 代码结构

代码共 5 个文件：

- ss_function.py: 快速幂、多项式构造、多项式求值、拉格朗日重构
- ss_student.py: 学生端一把私有数据拆成 3 份发给另外两方
- compute_local.py: 学生端一汇总收到的所有份额
- average_counter.py: 计票端一读两个聚合值，重构总和并算平均值
- run_test.py: 自动化测试脚本

4.2 核心函数（ss_function.py）

```

1 def quickpower(a, b, p):
2     """快速幂：计算  $a^b \bmod p$ """
3     a = a % p
4     ans = 1
5     while b != 0:

```

```

6         if b & 1:
7             ans = (ans * a) % p
8             b >>= 1
9             a = (a * a) % p
10        return ans
11
12 def restructure_polynomial(x, fx, t, p):
13     """拉格朗日插值：由 t 个点 (x[i], fx[i]) 恢复 f(0)"""
14     ans = 0
15     for i in range(t):
16         fx[i] = fx[i] % p
17         fxi = 1
18         for j in range(t):
19             if j != i:
20                 # L_i(0) = prod_{j!=i} (-x[j]) / (x[i] - x[j])
21                 fxi = (-1 * fxi * x[j]
22                      * quickpower(x[i] - x[j], p - 2, p)) % p
23         fxi = (fxi * fx[i]) % p
24         ans = (ans + fxi) % p
25     return ans

```

Listing 1: 快速幂与拉格朗日插值重构

4.3 学生端：秘密份额分发 (ss_student.py)

```

1 id = int(input("请输入参与方id (1/2/3): "))
2 s = int(input(f"请输入 student_{id} 的私有数据 s: "))
3
4 # 构造一次多项式 f(x) = s + r*x, t-1=1 个随机系数
5 f = ss_f.get_polynomial(s, 1, p, str(id))
6
7 # 计算三份份额 f(1), f(2), f(3)
8 shares_x = [1, 2, 3]
9 shares_y = [ss_f.count_polynomial(f, x, p) for x in shares_x]
10
11 # 写入文件 student_{id}_{j}.txt, 发送给 Student_j
12 for j in range(1, 4):
13     with open(f"student_{id}_{j}.txt", "w") as fp:
14         fp.write(str(shares_y[j - 1]))

```

Listing 2: 学生端秘密份额分发

4.4 学生端：本地聚合 (compute_local.py)

```
1 id = int(input("请输入参与方id (1/2/3): "))
2
3 # 读取三方各给自己的份额: student_1_id, student_2_id, student_3_id
4 data = []
5 for i in range(1, 4):
6     with open(f"student_{i}_{id}.txt", "r") as f:
7         data.append(int(f.read()))
8
9 # 聚合: d_id = sum(所有份额) mod p, 利用同态加法性质
10 d = sum(data) % p
11
12 with open(f"d_{id}.txt", "w") as f:
13     f.write(str(d))
14 print(f"Student_{id} 聚合值 d_{id} = {d}")
```

Listing 3: 学生端本地聚合

4.5 计票端：平均值计算 (average_counter.py)

```
1 # 读取任意两个学生的聚合值 (取学生2和学生3)
2 x_vals = [2, 3]
3 d_vals = [int(open(f"d_{i}.txt").read()) for i in x_vals]
4
5 # 拉格朗日插值恢复  $F(0) = s_1 + s_2 + s_3$ 
6 total = ss_f.restructure_polynomial(x_vals, d_vals, 2, p)
7 print(f"重构总和: {total}")
8
9 # 平均值 = total *  $3^{-1} \bmod p$ 
10 inv_3 = pow(3, p - 2, p) # 费马小定理求3的乘法逆元
11 avg = (total * inv_3) % p
12 print(f"平均值: {avg}")
```

Listing 4: 计票端平均值计算

5 运行结果

运行测试脚本：

```
1 conda activate new
2 cd d:\大三下\数据安全\Lab3
3 python run_test.py
```

测试了三组数据：常规数据（10,20,30）、含 0 数据（0,15,30）、较大数据（100,200,300）。脚本按协议顺序执行三个步骤，输出如图 3 所示。

```
D:\大三下\数据安全>D:/30515/Anaconda/envs/new/python.exe d:/大三下/数据安全/Lab3/run_test.py
```

```
=====  
测试用例：常规数据
```

```
输入：Student1=10, Student2=20, Student3=30  
预期平均值：20  
=====
```

【步骤1】秘密份额分发

```
Student1: f1(x)=10+629094632x 份额=(629094642, 258189267, 887283899)  
Student2: f2(x)=20+470101453x 份额=(470101453, 940202886, 410304312)  
Student3: f3(x)=30+340339897x 份额=(340339927, 680679824, 21019714)
```

【步骤2】本地聚合

```
Student1: d_1 = 629094642+470101453+340339927 mod p = 439536015  
Student2: d_2 = 258189267+940202886+680679824 mod p = 879071970  
Student3: d_3 = 887283899+410304312+21019714 mod p = 318607918
```

【步骤3】计票端 - 拉格朗日插值重构

```
取 d_2=879071970, d_3=318607918  
重构总和 F(0) = 60  
平均值 = 60 * 3^(-1) mod p = 20
```

```
✓ 通过！ 平均值 = 20 (预期 20)  
=====
```

```
=====  
测试用例：含0数据
```

```
输入：Student1=0, Student2=15, Student3=30  
预期平均值：15  
=====
```

【步骤1】秘密份额分发

```
Student1: f1(x)=0+994784898x 份额=(994784898, 989569789, 984354680)  
Student2: f2(x)=15+714883818x 份额=(714883833, 429767644, 144651455)  
Student3: f3(x)=30+245560864x 份额=(245560894, 491121758, 736682622)
```

【步骤2】本地聚合

```
Student1: d_1 = 994784898+714883833+245560894 mod p = 955229618  
Student2: d_2 = 989569789+429767644+491121758 mod p = 910459184  
Student3: d_3 = 984354680+144651455+736682622 mod p = 865688750
```

【步骤3】计票端 - 拉格朗日插值重构

```
取 d_2=910459184, d_3=865688750  
重构总和 F(0) = 45  
平均值 = 45 * 3^(-1) mod p = 15
```

```
✓ 通过！ 平均值 = 15 (预期 15)  
=====
```

```
=====  
测试用例：较大数据
```

```
输入：Student1=100, Student2=200, Student3=300  
预期平均值：200  
=====
```

【步骤1】秘密份额分发

```
Student1: f1(x)=100+238361132x 份额=(238361232, 476722364, 715083496)  
Student2: f2(x)=200+942950476x 份额=(942950676, 885901145, 828851614)  
Student3: f3(x)=300+467119809x 份额=(467120109, 934239918, 401359720)
```

【步骤2】本地聚合

```
Student1: d_1 = 238361232+942950676+467120109 mod p = 648432010  
Student2: d_2 = 476722364+885901145+934239918 mod p = 296863413  
Student3: d_3 = 715083496+828851614+401359720 mod p = 945294823
```

【步骤3】计票端 - 拉格朗日插值重构

```
取 d_2=296863413, d_3=945294823  
重构总和 F(0) = 600  
平均值 = 600 * 3^(-1) mod p = 200
```

```
✓ 通过！ 平均值 = 200 (预期 200)
```

图 3: run_test.py 运行结果：秘密份额分发、本地聚合、平均值重构

步骤 1 中，每个学生生成随机一次多项式 $f_i(x) = s_i + r_i \cdot x$ 并输出三份秘密份额。步骤 2 中，每个学生汇总收到的 3 份份额，算聚合值 $d_j = f_1(j) + f_2(j) + f_3(j) \bmod p$ ，保存到 `d_j.txt`。步骤 3 中，计票员取 d_2, d_3 ，拉格朗日插值恢复 $F(0) = s_1 + s_2 + s_3 = 60$ ，再算：

$$\text{avg} = 60 \times 3^{-1} \bmod p = 60 \times 333333336 \bmod p = 20$$

三组测试均通过，最终输出平均值与预期一致。

正确性验证：计票员用 d_2, d_3 插值得到 $F(0) = 60$ ，再 $60 \times 3^{-1} \bmod p = 20$ 。整个过程 Student1 的数据 10 只以份额形式出现，计票员无法从中推断任何人的原始数据。

5.1 算法复杂度分析

阶段	操作	时间复杂度	说明
秘密分发	生成多项式	$O(n)$	$n = 3$ ，构造一次多项式
秘密分发	计算 n 个份额	$O(n)$	每个学生计算 3 份份额
本地聚合	读取文件并求和	$O(n)$	读取 3 个文件并相加
计票重构	拉格朗日插值	$O(t^2)$	$t = 2$ ，只需两个点
求平均值	模逆元与乘法	$O(\log p)$	费马小定理求 3^{-1}

通信复杂度：每个学生需要向其他两方发送份额，合计 $n(n-1) = 6$ 次传输。

6 安全性分析

6.1 秘密性

每个学生只收到其他学生发的一份份额，不知道随机系数 r_i ，因此无法还原他人秘密。单一份额 $f_i(j)$ 在 $\mathbb{Z}_p$ 上均匀分布，与 s_i 统计独立，少于 2 份份额不泄露任何信息。

6.2 正确性

多项式加法同态性保证 $F(x) = f_1(x) + f_2(x) + f_3(x)$ ， $F(0) = s_1 + s_2 + s_3$ 。聚合值 $d_j = F(j)$ 。拉格朗日插值定理保证任意 2 个点能唯一确定一次多项式 $F(x)$ ，精确恢复 $F(0)$ 。

6.3 隐私性

计票员拿到 d_2, d_3 （即 $F(x)$ 在两点处的值），能且仅能恢复 $F(0) = s_1 + s_2 + s_3$ （和平均值）。两个方程解不出三个未知数，单个学生数据受保护。

6.4 防恶意参与者

协议假设各方诚实执行。恶意发送假份额会导致 d_j 错误，最终总和错误。解决方法是引入验证性秘密共享（VSS），让每方能验证份额有效性。

安全性总结：

- 任何少于 2 个学生的聚合值都无法重构总和（(2, 3) 门限保证）
- 计票员重构的是三个数据的总和，无法推断单个数据
- 每个学生只接触自己的原始数据和来自他人的份额，不接触他人原始数据
- 随机多项式系数 r_i 保证了份额的不可预测性
- 当前协议假设诚实参与者，恶意参与者可通过 VSS 扩展防范

7 实验中遇到的问题

7.1 模数选取

一开始用小模数 p 测试，平均值结果和预期不符。原因是有限域里“除以 3”是乘法逆元，不是整数除法。选了大素数 $p = 1000000007$ 后解决。

7.2 负数取模

Python 的 $(-1) \bmod p = p - 1$ ，和数学上一致，但拉格朗日基函数里 $-x_j/(x_i - x_j)$ 项，当 $x_i - x_j < 0$ 时要先取模再求逆就会出错。解决办法是每次乘法结果统一取模，利用 Python 的非负取模行为处理符号。

8 总结

本次实验用 Shamir 秘密共享实现了安全多方平均值计算，协议结构参照教材 4.3.3（投票统计），只是把最后一步从恢复总票数改成求平均值。

核心是多项式加法的同态性：每方用随机多项式分享自己的秘密，各方收到份额后本地相加，聚合值恰好是总和多项式在对应点的值，计票员拉格朗日插值恢复总和后除以 3 就得到平均值。

通过实验理解了 (t, n) 门限秘密共享的构造原理、拉格朗日插值在有限域上的应用，以及怎么用多项式加法同态性设计安全多方计算协议。

附录：运行说明

环境要求

- Windows 11 操作系统
- Python 3.9 或更高版本
- 无需额外依赖库

运行步骤（自动化）

1. 激活 conda 环境: `conda activate new`
2. 进入目录并运行测试: `python run_test.py`

运行步骤（手动）

1. 三个学生分别运行 `ss_student.py`，依次输入 id (1/2/3) 和私有数据
2. 三个学生分别运行 `compute_local.py`，依次输入 id (1/2/3)，进行本地聚合
3. 计票员运行 `average_counter.py`，输出平均值

文件清单

- `ss_function.py` 一通用函数（快速幂、多项式构造与求值、拉格朗日插值）
- `ss_student.py` 一学生端秘密份额分发
- `compute_local.py` 一学生端本地聚合
- `average_counter.py` 一计票端平均值计算
- `run_test.py` 一自动化测试脚本